

RAG 检索增强生成系统设计

RAG 是目前最主流的 LLM 落地架构，它让大模型 "开卷考试"，而非仅凭记忆回答。

RAG 检索增强生成系统设计

RAG 是目前最主流的 LLM 落地架构，它让大模型 "开卷考试"，而非仅凭记忆回答。

- - -

一、RAG 基础原理

1.1 RAG 解决的核心问题

问题 说明 RAG 如何解决

知识截止 模型不知道训练后发生的事 从外部知识库检索最新信息

幻觉 模型在不确定时编造答案 基于检索到的文档生成，可溯源

私有数据 模型不包含企业内部数据 企业私有知识库可随时更新

领域知识 通用模型缺乏垂直领域深度 领域知识库提供专业信息

1.2 两条流水线

离线索引流水线（一次性预处理）：

原始文档 (P D F / W o r d / H T M L)

↓ 文档解析

结构化文本 (M a r k d o w n)

↓ 文档切分

文档块 (C h u n k s)

↓ E m b e d d i n g 向量化

文档向量 + 原文

↓ 写入数据库

向量数据库

在线查询流水线（每次请求执行）：

用户提问

↓ Embedding 向量化

查询向量

↓ 相似度检索

Top - K 候选文档

↓ [可选] 重排序

精选文档

↓ Prompt 组装

问题 + 检索文档 → LLM

↓

生成回答

- - -

二、文档解析详解

2.1 常见文档格式及挑战

格式 挑战 推荐方案

PDF 表格提取、多栏排版、扫描件 LlamaParse、pdfplumber、Unstructured

Word 样式丢失、图片位置 python-docx、mammoth

HTML 导航栏噪声、CSS 样式 BeautifulSoup、trafilatura

扫描件 需要OCR PaddleOCR、Tesseract + 多模态大模型

表格 结构化数据提取 Camelot、Tabula

PDF 解析的最佳实践

```
from langchaincommunity.documentloaders import PyPDFLoader
```

方式1：基础PDF解析

```
loader = PyPDFLoader("document.pdf")
```

```
pages = loader.load() # 每页一个Document对象
```

方式2：更智能的解析（保留表格和布局）

```
loader = UnstructuredPDFLoader(
```

```
"document.pdf",
```

```
mode="elements", # 按元素（标题、段落、表格）提取
```

```

strategy = " hires " , # 高精度模式（使用检测模型）
infer_table_structure = True # 推断表格结构
)
documents = loader.load()

```

方式3：使用LlamaParse（效果最好但需付费）

```

from llamaparse import LlamaParse
parser = LlamaParse(result_type="markdown")
documents = parser.loaddata("document.pdf")

- - -

```

三、文档切分（Chunking）详解

3.1 四种切分策略对比

策略 原理 优点 缺点 推荐场景

固定大小 按字符 / token数等分 实现简单 可能切断语义 日志、代码

递归字符 按分隔符优先级切分 保持语义完整 分隔符选择需调试 通用推荐

语义切分 用Embedding计算相邻句子相似度 自动发现语义转折 计算成本高 长文档、书籍

父子文档 小块检索+大块返回 兼顾精度和上下文 设计复杂 高质量场景

3.2 Chunk大小选择指南

场景 推荐大小 Overlap 原因

FAQ问答 256 - 512 tokens 50 短答案，小chunk更精准

技术文档 512 - 1024 tokens 100 需要完整的代码块 / 段落

长篇分析 1024 - 2048 tokens 200 需要更多上下文

表格数据 按行 / 逻辑单元 无 保持表格完整性

递归字符切分——最常用的切分方式

```

from langchain.textsplitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=512, # 每块最大token数

```

```

chunkoverlap=50, # 相邻块重叠的token数
separators=["\n\n", "\n", "。", ".", " ", ""], # 分
lengthfunction=len,
)

chunks = splitter.splittext(documenttext)
print(f"原始文档: {len(documenttext)} 字符")
print(f"切分为: {len(chunks)} 个文档块")
for i, chunk in enumerate(chunks[:3]):
    print(f"\n--- 块 {i+1} ({len(chunk)} 字符) ---")
    print(chunk[:200])

```

3.3 元数据保留

切分时保留元数据对后续检索和过滤至关重要：

```

from langchain.schema import Document

```

每个chunk应该包含的元数据

```

chunkswithmetadata = [
    Document(
        pagecontent=chunktext,
        metadata={
            "source": "usermanualv3.pdf", # 来源文件
            "page": 42, # 页码
            "section": "3.2 安装配置", # 章节
            "chunkid": "doc1chunk5", # 唯一ID
            "doctype": "table", # 内容类型
            "createdat": "2025-01-15", # 文档日期
        }
    )
]

- - -

```

四、Embedding 模型详解

4.1 模型选择指南

模型 维度 语言 MTEB排名 特点

text-embedding-3-small (OpenAI)	1536	多语言	—	性价比高, A
text-embedding-3-large (OpenAI)	3072	多语言	—	精度最高, 成
BAAI/bge-m3	1024	中英文	前3	中文效果最佳, 开源
BAAI/bge-large-zh-v1.5	1024	中文	—	中文专用, 效果好
sentence-transformers/all-MiniLM-L6-v2	384	英文	—	极
intfloat/multilingual-e5-large	1024	多语言	前5	多语言通用

使用开源Embedding模型

```
from sentence transformers import SentenceTransformer

model = SentenceTransformer('BAAI/bge-m3')
```

编码文档

```
docs = ["深度学习是机器学习的一个分支", "自然语言处理研究文本数据"]
embeddings = model.encode(docs, normalize_embeddings=True)
```

编码查询 (bge模型建议加前缀)

```
query = "什么是深度学习?"
query_embedding = model.encode(["Represent this sentence with a keyword"],
                                normalize_embeddings=True)
```

4.2 相似度计算

方法 公式 值域 特点

余弦相似度	$\cos(a, b) = \frac{a \cdot b}{\ a\ \ b\ }$	$[-1, 1]$	最常用, 只看方向
点积	$a \cdot b$	$(-\infty, +\infty)$	归一化向量时等同余弦
欧氏距离	$\ a - b\ $	$[0, +\infty)$	同时考虑方向和大小

余弦相似度计算

```
def cosinesimilarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
```

批量检索：查询与所有文档的相似度

```
def retrieve(queryembedding, docembeddings, topk=10):
    similarities = np.dot(docembeddings, queryembedding)
    topindices = np.argsort(similarities)[-topk:][:]
    return topindices, similarities[topindices]

- - -
```

五、向量数据库实战

5.1 向量数据库选型对比

数据库 类型 规模 特点 适用场景

FAISS 库 亿级 速度快、内存可控 嵌入式、本地开发

ChromaDB 轻量DB 百万级 简单易用、自动Embedding 原型开发、小项目

Milvus 分布式DB 十亿级 云原生、高性能 企业生产环境

Weaviate DB 亿级 GraphQL API、模块化 多模态检索

Qdrant DB 亿级 Rust实现、过滤强 需要复杂过滤

Pinecone 云服务 亿级 全托管、零运维 快速上线、无运维

Elasticsearch 搜索引擎 亿级 传统+向量混合 已有ES基础设施

pgvector PG扩展 百万级 SQL查询、事务支持 已有PostgreSQL

选型决策树：

是否需要分布式？

├── 否（单机）→ FAISS（嵌入式）或 ChromaDB（轻量服务）

└── 是（分布式）

├── 已有基础设施？

| ├── Elasticsearch → ES 8.x kNN

| └── PostgreSQL → pgvector

└── 新项目

├── 需要全托管？→ Pinecone

└── 自建部署？→ Milvus（性能优先）或 Qdrant（轻量优先）

5.2 FAISS 详解

```
import faiss
import numpy as np
```

构建索引

```
dimension = 1024 # bge-m3的维度
ndocs = 100000
```

生成模拟数据

```
docembeddings = np.random.randn(ndocs, dimension)
faiss.normalizeL2(docembeddings) # L2归一化
```

选择索引类型

Flat： 精确搜索，适合小数据集

```
indexflat = faiss.IndexFlatIP(dimension) # 内积搜索（归一化后）
```

IVF： 倒排索引，适合大数据集

```
nlist = 100 # 聚类数
quantizer = faiss.IndexFlatIP(dimension)
indexivf = faiss.IndexIVFFlat(quantizer, dimension, nlist)
indexivf.train(docembeddings) # 需要训练
indexivf.add(docembeddings)
```

HNSW： 图索引，速度快，内存占用高

```
indexhnswnsw = faiss.IndexHNSWFlat(dimension, 32) #
```

搜索

```
query = np.random.randn(1, dimension).astype('float32')
faiss.normalizeL2(query)
D, I = indexflat.search(query, k=5) # 返回距离和索引
```

FAISS索引选择指南：

数据量 推荐索引 特点

< 100K IndexFlatIP 精确搜索，速度快

100K - 10M IndexIVFFlat 近似搜索，速度 / 精度平衡

> 10M IndexIVFPQ 量化压缩, 内存更小
任意 IndexHNSW 图搜索, 延迟最低

5.3 Milvus (企业级方案)

```
from pymilvus import connections, Collection, Field
```

连接Milvus

```
connections.connect(host="localhost", port="19530")
```

定义Schema

```
fields = [  
    FieldSchema(name="id", dtype=DataType.INT64, is_primary=True),  
    FieldSchema(name="embedding", dtype=DataType.FLOAT_VECTOR, dim=128),  
    FieldSchema(name="text", dtype=DataType.VARCHAR, max_length=1024),  
    FieldSchema(name="source", dtype=DataType.VARCHAR, max_length=1024),  
    FieldSchema(name="page", dtype=DataType.INT32),  
]  
schema = CollectionSchema(fields, description="知识库")
```

创建集合

```
collection = Collection("knowledgebase", schema)
```

创建索引 (HNSW)

```
indexparams = {  
    "metric_type": "COSINE",  
    "index_type": "HNSW",  
    "params": {"M": 16, "efConstruction": 256}  
}  
collection.create_index(fieldname="embedding", index_params=indexparams)
```

插入数据

```
data = [  
    embeddings, # List[List[float]]  
    texts, # List[str]  
    sources, # List[str]
```



```
pages, # List[int]
]
collection.insert(data)
```

搜索

```
collection.load()
searchparams = {"metrictype": "COSINE", "params":
results = collection.search(
data=[queryembedding],
annsfield="embedding",
param=searchparams,
limit=5,
expr='source == "usermanualv3.pdf"', # 元数据过滤
outputfields=["text", "source", "page"]
)
```

5.4 ChromaDB (轻量级方案)

```
import chromadb
```

初始化客户端

```
client = chromadb.PersistentClient(path=". /chroma
```

创建集合

```
collection = client.get_or_create_collection(
name="knowledgebase",
metadata={"hnsw:space": "cosine"} # 使用余弦相似度
)
```

添加文档

```
collection.add(
documents=["深度学习是机器学习的一个分支", "自然语言处理研究文本数据"],
metadatas=[{"source": "doc1"}, {"source": "doc2"}],
ids=["id1", "id2"]
)
```

检索

```
results = collection.query(
    querytexts = [ "什么是深度学习?" ],
    nresults = 5,
    where = { "source": "doc1" } # 可选：元数据过滤
)

- - -
```

六、高级 Embedding 技术

6.1 ColBERT——迟交互模型

传统 Bi-Encoder 将整个文档压缩为一个向量，不可避免地丢失细节信息。ColBERT 保留每个 token 的向量，检索时逐 token 计算最大相似度再求和。

Bi-Encoder:

文档 "深度学习是机器学习的分支" → [1 个向量, 1024 维]

查询 → [1 个向量, 1024 维]

相似度 = $\cos(\text{查询向量}, \text{文档向量})$

问题：一个向量无法表达文档的所有语义细节

ColBERT (Late Interaction):

文档 → [6 个 token 向量, 128 维] = 矩阵 D [6×128]

查询 → [4 个 token 向量, 128 维] = 矩阵 Q [4×128]

相似度 = $\sum_i \max_j \cos(q_i, d_j)$

对查询的每个 token，找文档中最相关的 token，求和

优势：精度显著提升（MTEB 上比 Bi-Encoder 高 5 - 10%）

劣势：存储成本高（每个文档存 N 个向量而非 1 个）

使用 RAGatouille (ColBERT 的易用封装)

```
from ragatouille import RAGPretrainedModel
```

加载 ColBERT 模型

```
model = RAGPretrainedModel.frompretrained("colbert-cc/colbert-cc-v1.0")
```

索引文档

```
model.index(  
    collection=documents, # List[str]  
    documentmetadatas=metadata,  
    indexname="knowledgebase"  
)
```

检索

```
results = model.search(query="什么是深度学习", k=5)
```

6.2 多向量表示 (Multi-Vector)

一篇文章可能涵盖多个主题，单一向量无法完整表示：

方案1：多 chunk 表示

文档 → 切分为多个 chunk → 每个 chunk 一个向量

检索时匹配任意 chunk

方案2：摘要向量 + 细节向量

文档 → 摘要向量 (整体语义) + 段落向量 (细节)

检索时先匹配摘要，再匹配段落

方案3：ColBERT 迟交互

文档 → 每个 token 一个向量

检索时逐 token 匹配

6.3 Embedding 模型微调

通用 Embedding 模型在你的领域可能表现不佳，微调可以显著提升：

```
from sentencetransformers import SentenceTransformer  
from torch.utils.data import DataLoader
```

加载基础模型

```
model = SentenceTransformer('BAAI/bge-m3')
```

准备训练数据：相关 query - doc 对

```
trainexamples = [
    InputExample(texts=["查询1", "相关文档1"], label=1.0),
    InputExample(texts=["查询1", "不相关文档"], label=0.0),
    # ... 更多数据
]
```

多负样本排序损失（效果最好）

```
traindataloader = DataLoader(trainexamples, shuffle=True)
trainloss = losses.MultipleNegativesRankingLoss(model)
```

微调

```
model.fit(
    trainobjectives=[(traindataloader, trainloss)],
    epochs=3,
    warmupsteps=100,
    outputpath=". / finetunedembedding"
)

- - -
```

七、混合检索详解

6.1 为什么需要混合检索？

检索方式 擅长 不擅长 示例

向量检索 语义匹配、容错 专有名词、精确匹配 "大模型" → "大型语言模型"

关键词检索 (BM25) 精确匹配、专有名词 语义理解、容错 "GPT-4" → "GPT-4"

最佳实践：两者融合，互补短板。

6.2 BM25 算法原理

$$\text{BM25}(D, Q) = \sum_{i=1}^n \text{IDF}(q_i) \cdot \left(\frac{f_i + k_1}{1 - b + b \cdot \frac{f_i}{\text{avgdl}}} \right)$$

```
from rankbm25 import BM25Okapi
```

BM25检索

```
tokenizedcorpus = [doc.split() for doc in documents]
bm25 = BM25Okapi(tokenizedcorpus)

query = "深度学习 模型"
tokenizedquery = query.split()
scores = bm25.getscores(tokenizedquery)
topindices = np.argsort(scores)[-5:][::-1]
```

6.3 融合策略

```
def hybridsearch(query, vectorstore, bm25, alpha=0.5):
    """
```

混合检索：向量检索 + BM25关键词检索

```
alpha: 向量检索权重 (1 - alpha为BM25权重)
    """
```

```
# 向量检索
```

```
vectorresults = vectorstore.similaritysearch(query, k=20)
vectorscores = {r.docid: r.score for r in vectorresults}
```

```
# BM25检索
```

```
bm25results = bm25.gettopn(query, k=20)
bm25scores = {r.docid: r.score for r in bm25results}
```

```
# 归一化分数
```

```
alldocs = set(vectorscores.keys()) | set(bm25scores.keys())
maxvs = max(vectorscores.values()) if vectorscores else 0
maxbs = max(bm25scores.values()) if bm25scores else 0
```

```
# 加权融合（RRF或线性加权）
```

```
finalscores = {}
for docid in alldocs:
    vs = vectorscores.get(docid, 0) / maxvs
    bs = bm25scores.get(docid, 0) / maxbs
    finalscores[docid] = alpha * vs + (1 - alpha) * bs
```

```
# 返回Top - K
ranked = sorted(finalscores.items(), key=lambda x: x[1], reverse=True)
return ranked[:5]

- - -
```

七、重排序 (Reranking)

7.1 为什么需要重排序？

向量检索是双编码器 (Bi-Encoder)：问题和文档分别编码，只计算向量相似度，速度快但精度有限

重排序是交叉编码器 (Cross-Encoder)：将问题和文档一起输入模型，联合推理，精度高但速度慢

Bi-Encoder vs Cross-Encoder 对比：

Bi-Encoder (粗排)：

问题 → Encoder → [向量q] ←——cosine——→ [向量d] ← Encoder

优点：可预计算文档向量，检索速度快 $O(1)$

缺点：问题和文档没有交互，匹配精度有限

Cross-Encoder (精排)：

[CLS] 问题 [SEP] 文档 [SEP] → Encoder → 分数

优点：问题和文档深度交互，精度高

缺点：每对都要过一遍模型，速度慢 $O(N)$

两阶段检索架构是工程落地的标配方案：先用 Bi-Encoder 从百万文档中召回 Top - 50 ~ 100，再用 Cross-Encoder 精排到 Top - 5。

7.2 完整的重排序实现

```
from sentencetransformers import CrossEncoder
```

加载重排序模型

```
reranker = CrossEncoder("BAAI/bge-reranker-v2-m3")
```

```
def retrieve_with_reranking(query, vectorstore, top_k):
    """
```

粗排 → 精排 两阶段检索

"""

粗排：向量检索召回Top-N

```
candidates = vectorstore.similaritysearch(query,
```

精排：Cross-Encoder重排序

```
pairs = [[query, doc.pagecontent] for doc in cand
```

```
scores = reranker.predict(pairs)
```

按分数排序，返回Top-K

```
ranked = sorted(zip(scores, candidates), key=lambda
```

```
return [doc for , doc in ranked[:topk]]
```

7.3 常用重排序模型

模型 语言 大小 特点

BAAI/bge-reranker-v2-m3 多语言 568M 中文最佳，多语言支持

BAAI/bge-reranker-large 英文 560M 英文效果好

BAAI/bge-reranker-base 英文 278M 轻量级

cross-encoder/ms-marco-MiniLM-L-6-v2 英文 22M 极轻量，速

Jina-reranker-v2-base-multilingual 多语言 278M 支持长文档

7.4 重排序的进阶技巧

分数归一化：不同查询的绝对分数不可比，需要归一化：

```
def normalizescores(scores):
```

```
    """Min-Max归一化到[0,1]"""
```

```
    mins, maxs = scores.min(), scores.max()
```

```
    if maxs - mins < 1e-6:
```

```
        return np.oneslike(scores) * 0.5
```

```
    return (scores - mins) / (maxs - mins)
```

多查询重排序：一个复杂问题拆成多个子查询，分别检索再合并：

```
def multiqueryreranking(question, llm, vectorstor
```

```
" " "
```

M u l t i - Q u e r y : 让 L L M 生成多个视角的查询, 扩大召回

```
" " "
```

```
# L L M 生成多个改写查询
```

```
p r o m p t = f " " " 请从不同角度生成 3 个与以下问题相关的搜索查询:
```

```
原始问题: { q u e s t i o n }
```

```
查询 1 / 查询 2 / 查询 3: " " "
```

```
q u e r i e s = l l m . g e n e r a t e ( p r o m p t ) . s t r i p ( ) . s p l i t ( ' \ n '
```

```
a l l q u e r i e s = [ q u e s t i o n ] + q u e r i e s # 包含原始查询
```

```
# 对每个查询检索并去重
```

```
a l l c a n d i d a t e s = { }
```

```
f o r q i n a l l q u e r i e s :
```

```
c a n d i d a t e s = v e c t o r s t o r e . s i m i l a r i t y s e a r c h ( q , k = 2 0
```

```
f o r d o c i n c a n d i d a t e s :
```

```
i f d o c . p a g e c o n t e n t n o t i n a l l c a n d i d a t e s :
```

```
a l l c a n d i d a t e s [ d o c . p a g e c o n t e n t ] = d o c
```

```
# 用原始问题对候选文档重排序
```

```
p a i r s = [ [ q u e s t i o n , t e x t ] f o r t e x t i n a l l c a n d i d a t e s
```

```
s c o r e s = r e r a n k e r . p r e d i c t ( p a i r s )
```

```
r a n k e d = s o r t e d ( z i p ( s c o r e s , a l l c a n d i d a t e s . v a l u e s (
```

```
r e t u r n [ d o c f o r , d o c i n r a n k e d [ : t o p k ] ]
```

```
- - -
```

八、Advanced RAG 架构

8.1 查询优化

查询改写 (Q u e r y R e w r i t i n g) :

```
d e f q u e r y r e w r i t e ( o r i g i n a l q u e r y , l l m ) :
```

```
" " " 用 L L M 将口语化提问改写为更适合检索的查询 " " "
```

```
p r o m p t = f " " " 请将以下用户问题改写为更适合搜索引擎检索的关键词形式。
```


只输出改写后的查询，不要其他内容。

原始问题：{ originalquery }

改写后的查询：" " "

```
return llm.generate(prompt)
```

Step-Back Prompting：让LLM先回答一个更抽象的"后退问题"，获取背景知识：

```
def stepbacksearch(query, llm, vectorstore, topk=
" " "
```

Step-Back：先问一个更抽象的问题获取背景知识

再用背景知识+原问题一起检索

```
" " "
```

```
# 生成后退问题
```

```
stepbackprompt = f" " "请生成一个比以下问题更抽象、更一般化的背景问题：
```

原始问题：{ query }

背景问题：" " "

```
stepbackquery = llm.generate(stepbackprompt)
```

```
# 分别检索原问题和背景问题
```

```
originalresults = vectorstore.similaritysearch(qu
```

```
backgroundresults = vectorstore.similaritysearch(q
```

```
# 合并去重
```

```
alldocs = originalresults + backgroundresults
```

```
seen = set()
```

```
uniquedocs = []
```

```
for doc in alldocs:
```

```
if doc.pagecontent not in seen:
```

```
seen.add(doc.pagecontent)
```

```
uniquedocs.append(doc)
```

```
return uniquedocs[:topk-2]
```

HyDE（假设文档嵌入）：

```
def hydesearch(query, llm, vectorstore, topk=5):  
    """
```

让LLM先"猜"一个答案，用假设答案的向量去检索

假设答案通常比原问题包含更多专业术语，检索更准确

```
    """
```

```
# LLM生成假设答案
```

```
prompt = f"请简要回答以下问题：{query}"
```

```
hypotheticalanswer = llm.generate(prompt)
```

```
# 用假设答案（而非原问题）去检索
```

```
results = vectorstore.similaritysearch(hypotheticalanswer)
```

```
return results
```

8.2 CRAG（修正式RAG）

```
def cragsearch(query, llm, vectorstore, websearch):  
    """
```

CRAG：先评估检索结果质量，质量差则触发Web搜索补充

```
    """
```

```
# 检索
```

```
docs = vectorstore.similaritysearch(query, k=topk)
```

```
# 让LLM评估检索结果的相关性
```

```
evalprompt = f"""评估以下检索文档是否与问题相关。
```

问题：{query}

文档：{docs[0].pagecontent[:500]}

相关性评分（1 - 5）："""

```
score = int(llm.generate(evalprompt).strip())
```

```
if score >= 3:
```

```
# 检索结果足够好
```

```
return docs
```

```
else:
```

```
# 触发Web搜索补充
```

```
webresults = websearchtool.search(query)
return docs + webresults
```

8.3 Self-RAG (自反思RAG)

Self-RAG让模型自主决定是否需要检索、检索结果是否有用、生成是否忠实：

Self-RAG的三个反思点：

1. Retrieve：这个问题需要检索吗？（事实性问题→需要，创意性问题→不需要）
2. IsRelevant：检索到的文档与问题相关吗？
3. IsFaithful：生成的回答有文档支撑吗？（防止幻觉）

```
def selfrag(query, llm, vectorstore, topk=5):
    """Self-RAG：模型自主决定是否检索和验证"""
```

第一步：判断是否需要检索

```
judgeprompt = f"""判断以下问题是否需要从知识库检索信息来回答。
```

如果是事实性问题（需要具体信息）返回"YES"，

如果是创意性 / 推理性问题返回"NO"。

问题：{query}"""

```
needretrieval = "YES" in llm.generate(judgeprompt)
```

```
if not needretrieval:
```

直接生成

```
return llm.generate(f"请回答：{query}")
```

第二步：检索

```
docs = vectorstore.similaritysearch(query, k=topk)
```

```
context = "\n\n".join([doc.pagecontent for doc in docs])
```

第三步：生成+自我验证

```
genprompt = f"""基于以下参考文档回答问题。
```

如果文档中没有相关信息，请明确说明"根据现有文档无法回答"。

参考文档：{context}

问题：{query}

回答：" " "

```
answer = llm.generate(genprompt)
```

第四步：验证回答的忠实度

```
verifyprompt = f" " "验证以下回答是否被参考文档所支撑。
```

回答中的每个事实是否都能在文档中找到依据？

参考文档：{ context }

回答：{ answer }

是否忠实 (YES / NO) : " " "

```
isfaithful = "YES" in llm.generate(verifyprompt)
```

```
if not isfaithful:
```

```
answer += " \n \n⚠ 注意：以上回答的部分内容可能不在参考文档中。 "
```

```
return answer
```

8.4 RAG的Prompt工程

Prompt是RAG系统中最被低估的环节。好的Prompt决定了生成质量的上限：

好的RAG Prompt模板

```
RAGPROMPTTEMPLATE = " " "你是一个专业的知识助手。请基于以下参考文档来回答用户
```

规则

1. 只基于参考文档中的信息回答，不要使用自己的知识
2. 如果文档中没有相关信息，明确说明 " 根据现有文档无法回答 "
3. 引用具体的文档来源（如 " 根据文档第X段 . . . "）
4. 如果多个文档有矛盾，指出矛盾并分别说明
5. 不要编造文档中不存在的信息

参考文档

```
{ context }
```

用户问题

```
{ question }
```

回答

" " "

P r o m p t 优化的常见技巧：

技巧 说明 效果

引用来源 要求回答时标注出处 可溯源，减少幻觉

不确定声明 要求 " 无法回答 " 时明确说 减少幻觉

分步推理 先分析文档再回答 提高推理准确性

格式约束 指定输出格式（列表 / 表格） 输出更结构化

负面示例 给出不好的回答示例 避免常见错误

- - -

九、G r a p h R A G

9 . 1 传统 R A G 的局限

传统 R A G 将知识库当作独立的文本碎片，无法回答需要跨文档、多跳推理的复杂问题。

例如：" 公司 A 和公司 B 的创始人是什么关系？ " ——需要跨多个文档推理。

N a i v e R A G 的主要问题：

问题 原因 示例

多跳推理失败 答案分散在多个文档 " A 的创始人和 B 的 C T O 是同学吗？ "

全局性问题 需要汇总大量文档 " 所有文档的主要主题是什么？ "

实体关系缺失 只检索文本片段 " A 公司的投资方还投资了谁？ "

上下文碎片化 检索到的文档缺乏关联 同一实体的信息被分散

9 . 2 G r a p h R A G 架构

离线阶段：

文档 → L L M 提取三元组 → 存入图数据库（N e o 4 j）

在线阶段：

用户问题

|—— 向量检索 → 相关文档块

|—— 图检索 → 实体关系子图

↓

图文融合 → L L M生成

G r a p h R A G v s N a i v e R A G v s A d v a n c e d R A G :

维度 N a i v e R A G A d v a n c e d R A G G r a p h R A G

检索方式 向量检索 向量 + 关键词 + 重排 向量 + 图遍历

多跳推理 部分

全局性问题 (社区摘要)

实体关系

构建成本 低 中 高 (需建图)

适用场景 简单问答 专业问答 复杂推理

9 . 3 知识图谱构建

G r a p h R A G 的核心：知识图谱构建

```
def extracttriples(text, llm):
```

```
    """ 使用 L L M 从文本中提取知识三元组 """
```

```
    prompt = f """ 从以下文本中提取知识三元组（主语， 关系， 宾语）：
```

```
    文本：{ text }
```

```
    三元组列表： """
```

```
    response = llm.generate(prompt)
```

```
    # 解析输出，提取三元组
```

```
    triples = parsetriples(response)
```

```
    return triples
```

示例输出

```
( " O p e n A I " ,   " 创建了 " ,   " G P T - 4 " )
```

```
( " G P T - 4 " ,   " 属于 " ,   " 大语言模型 " )
```

```
( " 大语言模型 " ,   " 基于 " ,   " T r a n s f o r m e r " )
```

Microsoft GraphRAG 的社区摘要方法：

```
def buildcommunitysummaries(graph, llm):  
    """
```

Microsoft GraphRAG的核心创新：

1. 构建实体关系图
2. 用图聚类（Leiden算法）发现社区
3. LLM为每个社区生成摘要
4. 回答全局问题时汇总相关社区摘要

```
    """
```

```
    import networkx as nx  
    import community # python-louvain
```

```
# 1. 构建图
```

```
G = nx.Graph()  
for subj, rel, obj in triples:  
    G.addedge(subj, obj, relation=rel)
```

```
# 2. 社区检测
```

```
communities = community.bestpartition(G)
```

```
# 3. 为每个社区生成摘要
```

```
communitysummaries = {}  
for commid in set(communities.values()):  
    members = [n for n, c in communities.items() if c == commid]  
    subgraph = G.subgraph(members)
```

```
# 将子图转为文本描述
```

```
description = graphtotext(subgraph)
```

```
# LLM生成摘要
```

```
summary = llm.generate(f"请总结以下实体关系的核心内容：\n{description}")  
communitysummaries[commid] = summary
```

```
return communitysummaries
```

9.4 图检索策略

```
def graphenhancedsearch(query, vectorstore, graph):  
    """
```

图增强检索：同时利用向量检索和图遍历

```
    """
```

```
    # Step 1: 向量检索找到相关文档
```

```
    vectorresults = vectorstore.similaritysearch(query)
```

```
    # Step 2: 从检索结果中提取实体
```

```
    entityprompt = f"从以下文本中提取关键实体：\n{vectorresults[0]}
```

```
    entities = llm.generate(entityprompt).split(',')
```

```
    # Step 3: 图遍历获取相关子图
```

```
    subgraphtriples = []
```

```
    for entity in entities:
```

```
        # 查询该实体的所有关系（1跳或2跳）
```

```
        related = graphdb.query(  
            "MATCH (n) - [r] - (m) WHERE n.name = $entity RETURN  
            entity=entity  
        )
```

```
        subgraphtriples.extend(related)
```

```
    )
```

```
    subgraphtriples.extend(related)
```

```
    # Step 4: 将图信息注入上下文
```

```
    graphcontext = "\n".join([
```

```
        f"{t['n']} {t['r']} {t['m']}" for t in subgraphtriples  
    ])
```

```
    # Step 5: 生成回答
```

```
    prompt = f"""基于以下文档和知识图谱信息回答问题。
```

```
文档：{[doc.pagecontent for doc in vectorresults]}
```

```
知识图谱：{graphcontext}
```

```
问题：{query}"""
```



```
return llm.generate(prompt)
```

- - -

十、RAG 评估

10.1 RAGAS 框架

RAG 评估需要同时关注检索质量和生成质量两个维度：

指标 维度 含义 计算方式

Context Recall 检索 答案中的信息有多少被检索到 答案片段 vs 检索文档

Context Precision 检索 检索到的文档中相关比例 相关文档数 / 总检索数

Faithfulness 生成 答案是否有文档支撑 可溯源句数 / 总句数

Answer Relevance 生成 答案是否回答了问题 语义相似度

RAGAS 评估的直觉理解：

好的 RAG 系统 = 检得全 (Context Recall 高) + 检得准 (Context Precision 高)
+ 忠于文档 (Faithfulness 高) + 答对问题 (Answer Relevance 高)

常见问题模式：

- 检索不全 (Recall 低) → 回答不完整
- 检索不精 (Precision 低) → 噪声文档干扰生成
- 不忠实 (Faithfulness 低) → 幻觉，编造文档中没说的内容
- 答非所问 (Relevance 低) → 检索到的不相关或生成跑题

```
from ragas import evaluate
```

```
from ragas.metrics import faithfulness, answerrelevance
```

评估 RAG 系统

```
results = evaluate(  
    dataset=evaldataset,  
    metrics=[faithfulness, answerrelevance, contextrelevance]  
)  
print(results)
```

10.2 手动评估指标

```
def compute_faithfulness(answer, retrieved_docs):  
    """
```

手动计算忠实度：回答中的每个断言是否有文档支撑

```
    """
```

```
# Step 1: 从回答中提取断言
```

```
claims_prompt = f"从以下回答中提取所有事实断言：\n{answer}"
```

```
claims = llm.generate(claims_prompt).split('\n')
```

```
# Step 2: 检查每个断言是否有文档支撑
```

```
supported = 0
```

```
for claim in claims:
```

```
    for doc in retrieved_docs:
```

```
        verify_prompt = f"""以下断言是否被文档所支撑？
```

```
断言：{claim}
```

```
文档：{doc}
```

```
回答YES或NO："""
```

```
        if "YES" in llm.generate(verify_prompt):
```

```
            supported += 1
```

```
        break
```

```
    return supported / len(claims) if claims else 0
```

```
def compute_context_relevance(question, retrieved_docs):
```

```
    """计算检索相关性：检索到的文档中与问题相关的比例"""
```

```
    relevant = 0
```

```
    for doc in retrieved_docs:
```

```
        judge_prompt = f"""以下文档是否与问题相关？
```

```
问题：{question}
```

```
文档：{doc.page_content[:500]}
```

```
相关 (YES / NO)："""
```

```
        if "YES" in llm.generate(judge_prompt):
```

```
            relevant += 1
```

```
return relevant / len(retrieveddocs)
```

10.3 A / B测试与持续优化

RAG系统的评估不应只看离线指标，还需要在线A / B测试：

A / B测试框架

```
class RAGABTest:
    def init(self, systema, systemb):
        self.systema = systema # 对照组
        self.systemb = systemb # 实验组
        self.results = {'a': [], 'b': []}

    def evaluatepair(self, question, referenceanswer):
        """对同一个问题，两个系统分别回答"""
        answera = self.systema.query(question)
        answerb = self.systemb.query(question)

        # 用LLM作为裁判
        judgeprompt = f"""请比较以下两个回答的质量。
问题：{question}
参考答案：{referenceanswer}
回答A：{answera}
回答B：{answerb}

哪个回答更好（A / B / Tie）？为什么？"""

        result = llm.generate(judgeprompt)
        return result

    def runbatch(self, questions, references):
        """批量评估"""
        for q, ref in zip(questions, references):
            result = self.evaluatepair(q, ref)
            # 记录结果...
```

RAG优化路线图：

Level 1: Naive RAG (基础可用)

→ 基本向量检索 + LLM生成

→ 预期: Faithfulness ~60 - 70%

Level 2: Advanced RAG (工程优化)

→ 混合检索 + Reranking + 查询优化

→ 预期: Faithfulness ~75 - 85%

Level 3: Modular RAG (模块化设计)

→ Self-RAG + CRAG + 多路召回

→ 预期: Faithfulness ~85 - 90%

Level 4: GraphRAG (知识图谱增强)

→ 图检索 + 社区摘要 + 多跳推理

→ 预期: 多跳问题准确率大幅提升

- - -

十一、完整 RAG 系统实现

端到端 RAG 系统

```
from langchain.vectorstores import Chroma
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.textsplitter import RecursiveCharacterTextSplitter
from langchain.documentloaders import DirectoryLoader
from sentencetransformers import CrossEncoder

class RAGSystem:
    def __init__(self, persist_dir="./chromadb", embedding_model="sentence-transformers/all-MiniLM-L6-v2", reranker_model="BAAI/bge-reranker-v1.1-zh-v1.1", splitter_model="RecursiveCharacterTextSplitter", chunk_size=512, chunk_overlap=50, separators=["\n\n", "\n", "。", ".", " ", ""]):
        self.persist_dir = persist_dir
        self.embedding_model = embedding_model
        self.reranker_model = reranker_model
        self.splitter_model = splitter_model
        self.chunk_size = chunk_size
        self.chunk_overlap = chunk_overlap
        self.separators = separators
```

```

self.vectorstore = Chroma(
    persistdirectory=persistdir,
    embeddingfunction=self.embeddings
)

def indexdocuments(self, filepath):
    """索引文档"""
    loader = DirectoryLoader(filepath, glob="/.md")
    docs = loader.load()
    chunks = self.splitter.splitdocuments(docs)
    self.vectorstore.adddocuments(chunks)

def query(self, question, topk=5, usereranking=True):
    """查询"""
    # 检索
    if usereranking:
        candidates = self.vectorstore.similaritysearch(question)
        pairs = [[question, doc.pagecontent] for doc in candidates]
        scores = self.reranker.predict(pairs)
        ranked = sorted(zip(scores, candidates), key=lambda x: x[0])
        docs = [doc for _, doc in ranked[:topk]]
    else:
        docs = self.vectorstore.similaritysearch(question, topk=topk)

    # 生成(这里简化, 实际用LLM)
    context = "\n\n".join([doc.pagecontent for doc in docs])
    return context

```

使用

```

rag = RAGSystem()
rag.indexdocuments("./docs")
result = rag.query("什么是Transformer?")

- - -

```

十二、RAG v s 微调选择指南

维度 RAG 微调

知识更新 即时（更新知识库） 需要重新训练

数据需求 文档即可 需要标注数据

可解释性 高（可溯源） 低

成本 低 高

风格适配 有限 强

幻觉控制 好（基于文档） 一般

推荐：先用 RAG，如果 RAG 无法解决风格 / 语气问题，再加微调。两者可以结合。

- - -

十三、多模态 RAG

传统 RAG 只处理文本，但现实中的知识库通常包含图像、表格、视频等多模态内容。

13.1 多模态文档解析

多模态文档的挑战：

PDF 文档可能包含：文字、图片、表格、公式、图表

每种内容需要不同的解析策略：

	标题（文本）	→	直接提取	
	正文（文本）	→	直接提取	
	表格	→	结构化提取	
	图片	→	多模态模型描述	
	公式	→	LaTeX 提取	
	图表	→	数据 + 描述	

多模态文档解析流水线

```
class MultiModalParser:
    """ 解析包含文本、表格、图片的复杂文档 """
```

```

def parsedocument(self, pdfpath):
    from unstructured.partition.pdf import partitionpdf

    # 按元素类型提取
    elements = partitionpdf(
        pdfpath,
        strategy="hires",
        infer_table_structure=True,
        extract_images_in_pdf=True, # 提取图片
        extract_image_block_types=["Image", "Table"]
    )

    results = []
    for element in elements:
        if element.category == "Image":
            # 用多模态LLM描述图片
            description = self.describeimage(element.metadata)
            results.append({
                "type": "image",
                "content": description,
                "originalpath": element.metadata.imagepath
            })
        elif element.category == "Table":
            # 表格转HTML/Markdown
            results.append({
                "type": "table",
                "content": element.metadata.text_as_html,
            })
        else:
            results.append({
                "type": "text",
                "content": str(element),
            })

```

```

return results

def describeimage(self, imagepath):
    """用多模态LLM描述图片内容"""
    import base64
    from openai import OpenAI
    client = OpenAI()

    with open(imagepath, "rb") as f:
        imagedata = base64.b64encode(f.read()).decode()

    response = client.chat.completions.create(
        model="gpt-4o",
        messages=[{
            "role": "user",
            "content": [
                {"type": "text", "text": "请详细描述这张图片的内容，包括所有文字、数据和"},
                {"type": "imageurl", "imageurl": {"url": f"data:image/jpeg;base64,{imagedata}"}}
            ]
        }]
    )
    return response.choices[0].message.content

```

13.2 图文混合索引与检索

```

class MultiModalRAG:
    """图文混合RAG系统"""

    def __init__(self):
        self.textstore = Chroma(persistdirectory="./textstore")
        self.imagestore = Chroma(persistdirectory="./imagestore")

    def indexdocument(self, elements):
        """索引多模态文档"""
        for elem in elements:

```



```

if elem["type"] == "text":
    self.textstore.add(documents=[elem["content"]], i
elif elem["type"] == "image":
    # 用CLIP或多模态模型编码图像
    imageembedding = self.encodeimage(elem["originalp
    self.imagestore.add(
        embeddings=[imageembedding],
        documents=[elem["content"]], # 图片的文本描述
        ids=[elem["id"]]
    )

def query(self, question, topk=5):
    """混合检索文本和图片"""
    textresults = self.textstore.similaritysearch(que
    imageresults = self.imagestore.similaritysearch(o

# 合并、去重、排序
allresults = textresults + imageresults
# ... 合并逻辑
return allresults

- - -

```

十四、结构化数据 RAG

14.1 Text-to-SQL RAG

当知识库是关系型数据库时，需要将自然语言转为SQL查询：

```

class TextToSQLRAG:
    """自然语言 → SQL → 查询 → LLM总结"""

    def init(self, dbconnection, llm):
        self.db = dbconnection
        self.llm = llm

    def getschemainfo(self):

```

```
""" 获取数据库Schema信息 """
```

```
# 提取表名、列名、类型、外键关系
```

```
schemaprompt = """
```

数据库包含以下表：

```
- orders: id, userid, productid, amount, createda
```

```
- users: id, name, email, city
```

```
- products: id, name, category, price
```

关系: orders.userid → users.id, orders.productid → p

```
"""
```

```
return schemaprompt
```

```
def query(self, question):
```

```
""" 自然语言查询数据库 """
```

```
# Step 1: 生成SQL
```

```
sqlprompt = f""" 基于以下数据库Schema，将用户问题转换为SQL查询。
```

只输出SQL，不要其他内容。

```
Schema: {self.getschemainfo() }
```

```
用户问题: {question}
```

```
SQL: """
```

```
sql = self.llm.generate(sqlprompt)
```

```
# Step 2: 执行SQL
```

```
import pandas as pd
```

```
results = pd.readsql(sql, self.db)
```

```
# Step 3: LLM总结结果
```

```
summaryprompt = f""" 基于以下查询结果回答用户问题。
```

```
用户问题: {question}
```

```
SQL查询: {sql}
```

```
查询结果: {results.tostring() }
```

```
回答: """
```

```
return self.llm.generate(summaryprompt)
```

14.2 表格数据 RAG

```
class TableRAG:
```

```
""" 处理表格数据的 RAG """
```

```
def indextable(self, df, tablename):
```

```
""" 将DataFrame索引为可检索的文档 """
```

```
documents = []
```

```
# 方式1： 按行索引（适合查询具体记录）
```

```
for _, row in df.iterrows():
```

```
doctext = f"表{tablename}: " + ", ".join(
    f"{col}={val}" for col, val in row.items()
)
```

```
documents.append(doctext)
```

```
# 方式2： 按摘要索引（适合查询统计信息）
```

```
summary = f"表{tablename}包含{len(df)}行数据。" + \
```

```
f"列: {' , '.join(df.columns)}。" + \
```

```
f"统计: {df.describe().tostring()}"
```

```
documents.append(summary)
```

```
# 方式3： 转为自然语言描述
```

```
for col in df.selectdtypes(include=['object']).columns:
```

```
uniquevals = df[col].unique()[ :20]
```

```
doctext = f"表{tablename}的{col}列包含: {' , '.join(map(
```

```
documents.append(doctext)
```

```
return documents
```

```
---
```

十五、Agentic RAG

将Agent与RAG结合，让系统自主决定何时检索、检索什么、如何组合：

15.1 从被动检索到主动检索

传统RAG（被动）：

用户问题 → 检索 → 生成 → 返回

Agentic RAG（主动）：

用户问题 → Agent分析需要什么信息

→ 检索信息A → 判断是否足够

→ 不够 → 检索信息B → 判断

→ 需要计算 → 调用计算器工具

→ 需要最新数据 → 调用搜索工具

→ 综合所有信息 → 生成回答

15.2 ReAct + RAG 实现

```
class AgenticRAG:
```

```
    """基于ReAct模式的Agentic RAG"""
```

```
    def __init__(self, llm, vectorstore, tools=None):
```

```
        self.llm = llm
```

```
        self.vectorstore = vectorstore
```

```
        self.tools = tools or {}
```

```
    def run(self, question, maxiterations=5):
```

```
        """ReAct循环：思考→行动→观察→重复"""
```

```
        thoughthistory = []
```

```
        for i in range(maxiterations):
```

```
            # Thought: 分析当前状态，决定下一步
```

```
            thoughtprompt = f"""你是一个研究助手。请分析当前状态并决定下一步行动。
```

```
            问题: {question}
```

```
            已有信息: {thoughthistory}
```

```
            可选行动:
```

1. `search[查询词]` - 从知识库检索信息
2. `websearch[查询词]` - 从网络搜索
3. `calculate[表达式]` - 执行计算
4. `answer[答案]` - 给出最终回答

请先思考 (Thought), 再选择行动 (Action): " " "

```
response = self.llm.generate(thoughtprompt)
thoughthistory.append(response)

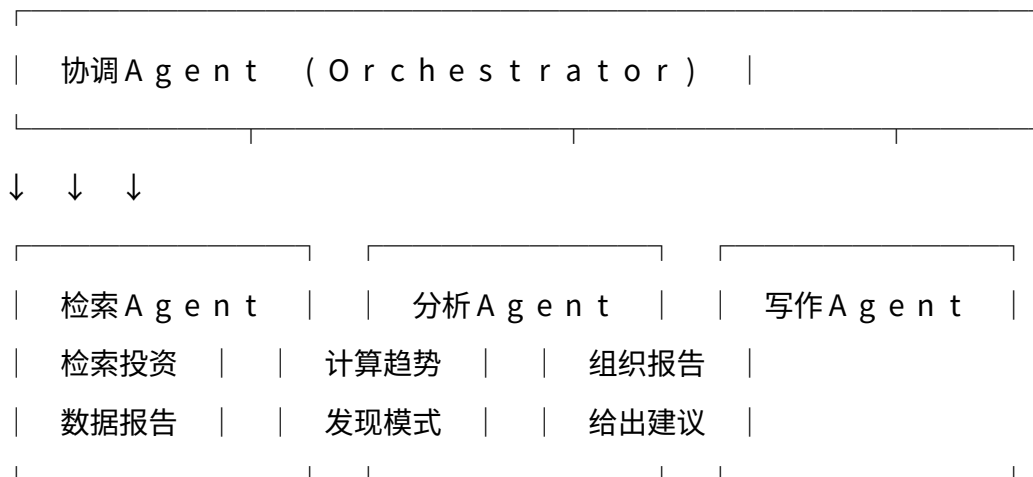
# 解析行动
if "answer[" in response.lower():
# 提取最终答案
return self.extractanswer(response)
elif "search[" in response:
query = self.extractaction(response, "search")
results = self.vectorstore.similaritysearch(query)
observation = "\n".join([doc.pagecontent for doc in results])
thoughthistory.append(f"Observation: {observation}")
elif "websearch" in response:
query = self.extractaction(response, "websearch")
results = self.tools["websearch"].run(query)
thoughthistory.append(f"Observation: {results}")
elif "calculate[" in response:
expr = self.extractaction(response, "calculate")
result = eval(expr) # 实际使用时应该用安全的方式
thoughthistory.append(f"Observation: {result}")

return "无法在限定步数内完成任务"
```

15.3 多智能体 RAG

复杂查询的分治策略：

用户： "分析2024年AI行业投资趋势并给出建议"



每个Agent有专属的：

- 检索源（不同的知识库 / 数据库）
- 工具集（搜索 / 计算 / 画图）
- Prompt 模板

- - -

十六、RAG 生产化

16.1 增量索引与更新

```

class IncrementalIndexer:
    """ 增量索引管理器——避免每次重建全量索引 """

    def init(self, vectorstore, doctracker):
        self.vectorstore = vectorstore
        self.tracker = doctracker # 记录文档的hash和索引状态

    def updateindex(self, documentdir):
        """ 增量更新索引 """
        import hashlib

        # Step 1: 扫描文档，计算hash
        currentdocs = {}
        for filepath in glob(f"{documentdir}/*/*", recursive=True):
            if filepath.endswith(('.md', '.pdf', '.txt')):

```

```

dochash = hashlib.md5(open(filepath, 'rb').read()).hexdigest()
currentdocs[filepath] = dochash

# Step 2: 与历史记录对比
trackeddocs = self.tracker.getall()

# 新增文档
newdocs = set(currentdocs.keys()) - set(trackeddocs.keys())
# 修改文档(hash变了)
modifieddocs = {
    path for path, hashval in currentdocs.items()
    if path in trackeddocs and trackeddocs[path] != hashval
}
# 删除文档
deleteddocs = set(trackeddocs.keys()) - set(newdocs.keys())

# Step 3: 增量操作
for path in deleteddocs:
    self.vectorstore.delete(filter={"source": path})

for path in newdocs | modifieddocs:
    chunks = self.parseandchunk(path)
    if path in modifieddocs:
        self.vectorstore.delete(filter={"source": path})
    self.vectorstore.adddocuments(chunks)

# Step 4: 更新追踪记录
self.tracker.update(currentdocs)

print(f"索引更新: +{len(newdocs)} 新增, ~{len(modifieddocs)} 删除")

```

16.2 缓存策略

```

import hashlib
import json
from functools import lru_cache

```

```

class RAGCache:
    """RAG查询缓存——相同 / 相似问题直接返回缓存结果"""

    def init(self, vectorstore, ttl=3600):
        self.cache = {} # 实际用Redis
        self.vectorstore = vectorstore
        self.ttl = ttl

    def getcachekey(self, question):
        """生成缓存key"""
        # 标准化问题（去空格、转小写）
        normalized = question.strip().lower()
        return hashlib.md5(normalized.encode()).hexdigest()

    def query(self, question, topk=5):
        cachekey = self.getcachekey(question)

        # 检查缓存
        if cachekey in self.cache:
            cached = self.cache[cachekey]
            if time.time() - cached['timestamp'] < self.ttl:
                print(f"缓存命中: {question}")
                return cached['result']

        # 缓存未命中, 执行检索
        result = self.vectorstore.similaritysearch(question)

        # 写入缓存
        self.cache[cachekey] = {
            'result': result,
            'timestamp': time.time()
        }

        return result

    def invalidate(self, sourcefile=None):

```



```

""" 使缓存失效（文档更新时调用） """
if source_file is None:
    self.cache.clear() # 全量失效
else:
    # 只失效与该文档相关的缓存（需要额外追踪）
    pass

```

16.3 并发与性能优化

RAG系统性能瓶颈分析：

用户请求 → [文档解析] → [Embedding 计算] → [向量检索] → [LLM生成]

CPU密集 GPU密集 内存 / IO GPU密集

可缓存 可缓存 可缓存 主要瓶颈

优化策略：

1. Embedding 缓存

相同文档的向量只需计算一次

用文档hash做key，缓存embedding结果

2. 批量Embedding

将多个文档 / 查询合并为一个batch

减少GPU kernel启动开销

3. 异步流水线

文档解析 → 异步Embedding → 异步写入向量库

用户请求 → 异步检索 → 异步LLM生成

4. 连接池管理

向量数据库连接池（避免频繁建连）

LLM推理连接池（vLLM本身就是高并发）

5. 预热策略

启动时预加载热点文档的embedding

预热LLM KV Cache（常见system prompt）

典型性能指标：

文档索引： ~ 100 docs / s (含 Embedding 计算)

向量检索： < 50 ms (百万级数据集)

Reranking： ~ 100 ms (50 个候选文档)

LLM生成： 1 - 10 s (取决于输出长度和模型大小)

16.4 生产环境踩坑经验

问题 原因 解决方案

检索不到相关文档 Query 与文档的语言风格差异大 查询改写、HyDE

检索到太多噪声 Top - K 太大或相似度阈值太低 降低 K 值、设相似度阈值、Reranking

表格数据检索差 表格转文本后语义丢失 专用表格解析 + 摘要索引

PDF 解析质量差 多栏、图片、公式无法正确提取 用 Llama Parse / 多模态模型

中英混合检索差 Embedding 模型中英文对齐不好 用 bge - m3 等支持多语言的模型

长文档答案不完整 Chunk 太小导致上下文断裂 父子文档策略、增大 Chunk

并发下延迟飙升 Embedding 计算 / LLM 推理成为瓶颈 缓存、批处理、扩容

索引更新延迟 全量重建太慢 增量索引、后台异步更新

- - -

十七、RAG 系统优化 workflow

RAG 系统优化的系统化方法论：

Step 1： 建立基线

- 用 RAGAS 评测当前系统
- 记录 Faithfulness , Relevance , Recall 指标
- 收集 Bad Case (用户反馈的回答不好的案例)

Step 2： 分析瓶颈

- 检索问题？
 - 召回率低 → 混合检索、多查询扩展
 - 精度低 → Reranking、调整 K 值
 - Query 不匹配 → 查询改写、HyDE
- 生成问题？

- | |—— 幻觉 → 加强 P r o m p t 约束、自验证
- | |—— 不完整 → 增加上下文、父子文档
- | |—— 格式差 → P r o m p t 工程、输出模板
- |—— 数据问题？
- |—— 解析差 → 换解析方案
- |—— 切分差 → 调整 c h u n k 策略
- |—— 覆盖不全 → 补充知识库

S t e p 3： 逐项优化

- |—— 每次只改一个变量
- |—— 记录改前改后的指标
- |—— 关注 B a d C a s e 是否改善

S t e p 4： A / B 测试

- |—— 线上分流对比
- |—— 关注用户满意度（比离线指标更重要）
- |—— 统计显著性检验

S t e p 5： 持续监控

- |—— 在线指标监控（延迟、错误率）
- |—— 定期 R A G A S 评测
- |—— 用户反馈闭环